it’s still good to understand how you might access performance information about Sidekiq without these tools.

Sidekiq’s Web UI Provides Observability at a Glance

You saw the Sidekiq Web UI earlier when learning about job failure and retries. The Web UI provides a lot more useful insights. Starting with Sidekiq 7, a “Metrics” tab can show you information about recent job executions. Sidekiq will retain these metrics for only eight hours, but it can provide a quick overview of how your jobs are doing at the moment. This is a great starting place for diagnosing a problem or ensuring the system is working properly.

At the volumes you’ve had in the example app, this tab won’t be very interesting, but if you have about 20 minutes, you can populate it with data like I did in the screenshots that follow. The most straightforward way to do this is to create Rake task and use Factory Bot (included in the example app for creating test data) to create fake orders and products.[12]

Create a Rake task in `lib/tasks/dev/load_sidekiq.rake` that looks like so:

```
require "factory_bot"

namespace :dev do
  desc "Load a bunch of jobs into Sidekiq"
  task load_sidekiq: :environment do
    products = 10.times.map { FactoryBot.create(:product) }
    1_000.times do |i|
      order = FactoryBot.create(:order, product: products.sample)
      CompleteOrderJob.perform_async(order.id)
      sleep(rand * 2)
    end
  end
end
```

To create more realistic job behavior, you should force your jobs to sleep randomly. For example, here is how to arrange for `CompleteOrderJob` to sleep some fractional number of seconds between 0 and 2: